Software Design Document

GPIO Driver Module

Lab 02 – Microcontrollers and Embedded Systems

Module	GPIO Driver
Version	1.1.0
Author(s)	J.M. Páez Sandoval, C.A. Martínez , D. Lopez Moreno, L.A. Lugo Muñiz
Institution	CETYS Universidad — Facultad de Ingeniería
Date	25/04/2026

1. Introduction

1.1 Purpose

This document describes the design and architecture of the gpio_driver module for Lab 2. It covers the updated implementation that adds full Alternate Function configuration, atomic BSRR-based pin writes, output type and speed control, and a GPIO_Status_t error-return type.

1.2 Scope

The gpio_driver is a low-level C module for the STM32F411RE. It uses CMSIS GPIO_TypeDef pointers and provides a clean API for configuring digital I/O, alternate functions (e.g. TIM1_CH1 on PA8), and atomic pin operations. It is used by the led, pwm, and timer modules.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
GPIO	General-Purpose Input/Output peripheral
CMSIS	Cortex Microcontroller Software Interface Standard
AF	Alternate Function — routes pin to a peripheral (e.g. TIM1_CH1)
MODER	Mode Register — 2 bits per pin: 00=Input, 01=Output, 10=AF, 11=Analog
OTYPER	Output Type Register — 0=Push-pull, 1=Open-drain
OSPEEDR	Output Speed Register — 2 bits per pin (slew rate)
PUPDR	Pull-Up/Pull-Down Register — 2 bits per pin
AFR	Alternate Function Register — AFRL (pins 0–7), AFRH (pins 8–15)
BSRR	Bit Set/Reset Register — atomic set (bits[15:0]) / reset (bits[31:16])
IDR	Input Data Register — read-only pin states
ODR	Output Data Register — XOR for toggle
GPIO_Status_t	Return type: GPIO_OK=0, GPIO_ERROR=1

1.4 Requirements

All requirements for this module are allocated in Docs/SRS/GPIO_Driver_SRS.md (provided by the instructor without modification).

2. System Architecture

2.1 High-Level Design

The gpio_driver sits between the application/middleware layer and the hardware registers. Peripheral clock is enabled automatically inside gpio_init().

Layer	Modules
Application	main.c, led.c, pwm.c — call gpio_driver APIs; no register access
GPIO Driver	All GPIO register operations isolated here
Hardware	STM32F411RE GPIO registers via CMSIS GPIO_TypeDef pointers

2.2 Module Interfaces

Public API declared in gpio_driver.h. Dependencies: stm32f411xe.h, stdint.h, stddef.h. No HAL.

3. Detailed Design

3.1 Class Diagram

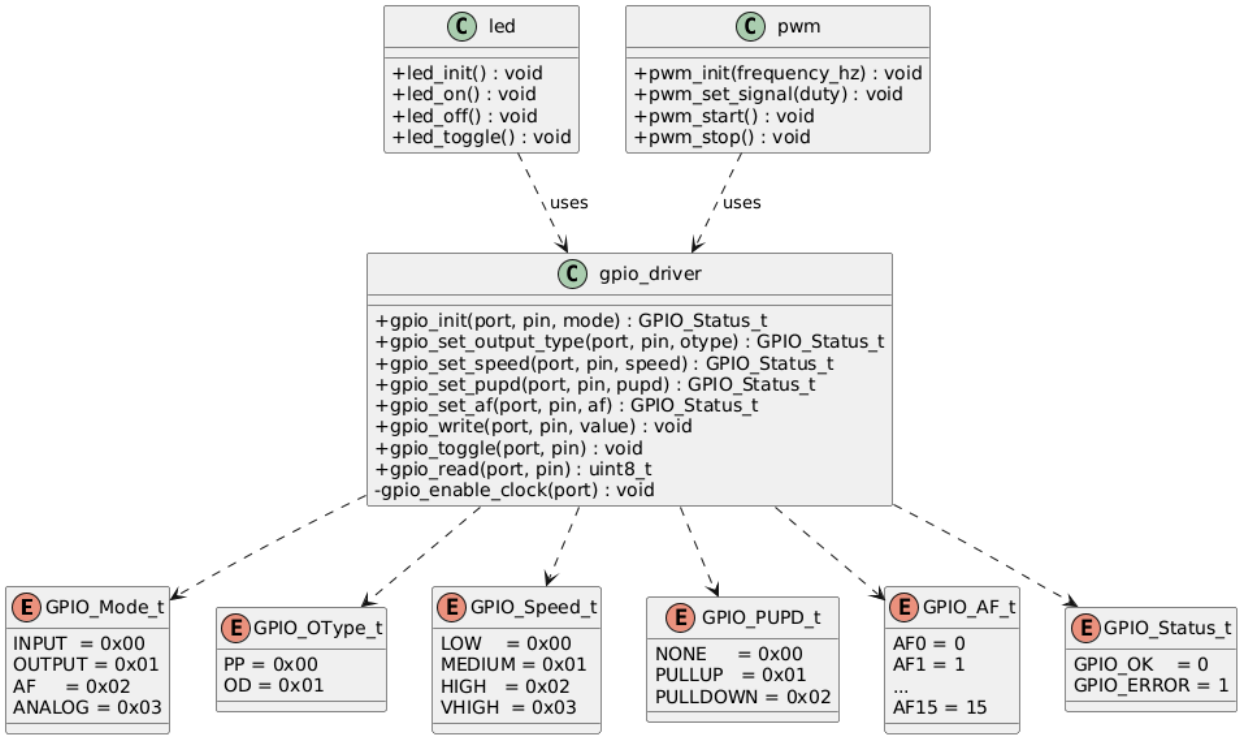

Figure 1. Class Diagram — gpio_driver module, enums, and dependent modules

3.2 Module Behavior

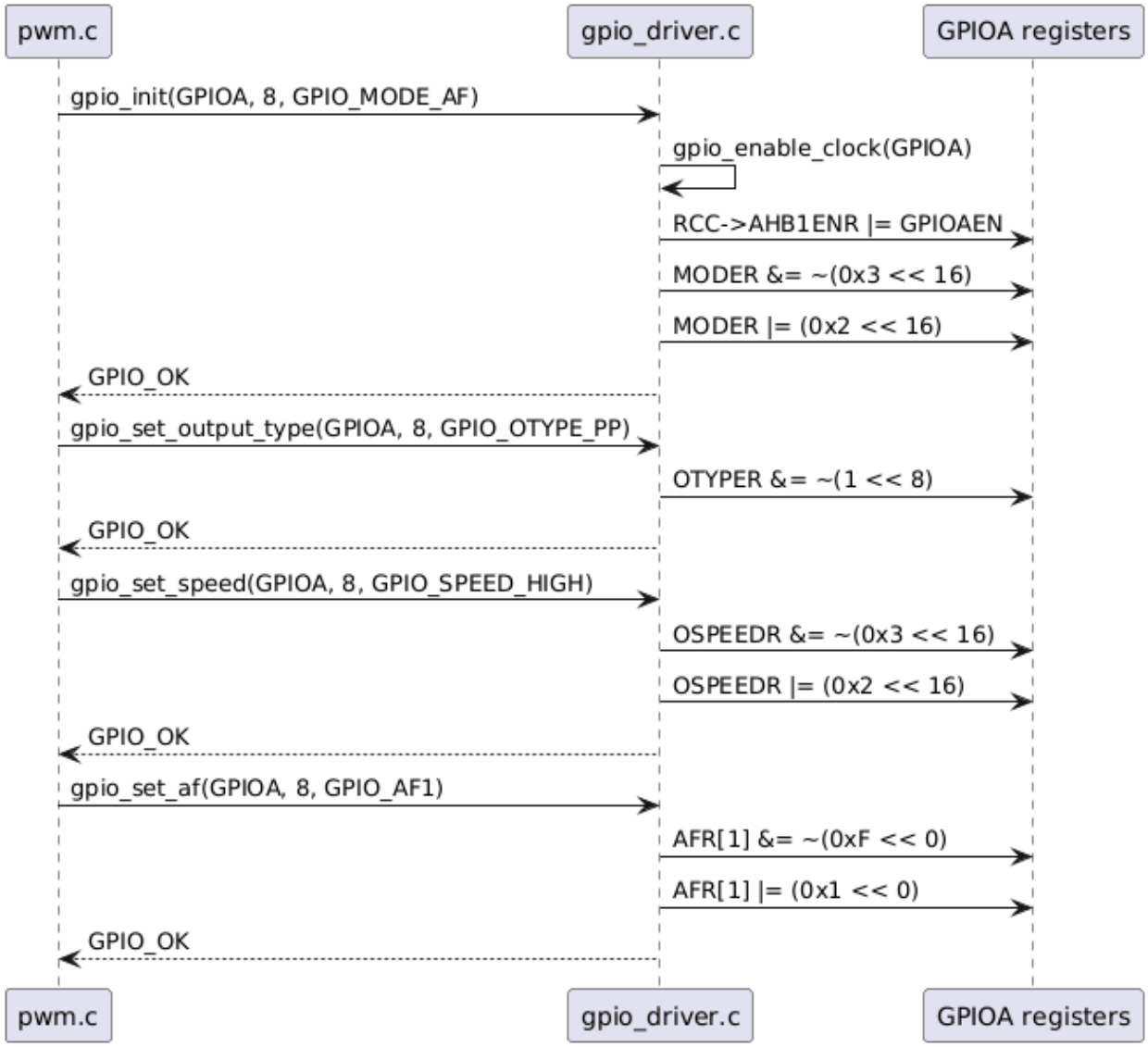

Figure 2. Sequence Diagram — Full pin initialization: gpio_init(), gpio_set_output_type(), gpio_set_speed(), gpio_set_af()

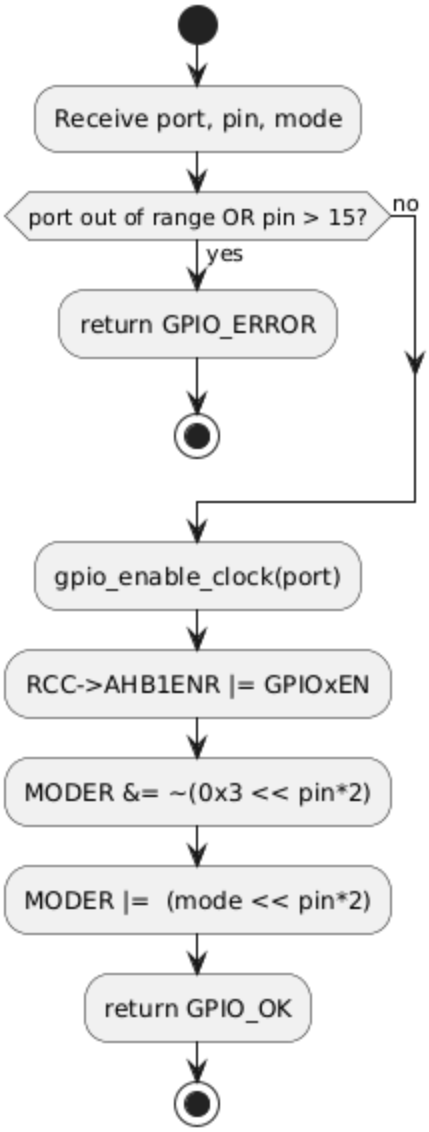

Figure 3. Activity Diagram — gpio_init() with NULL/range validation

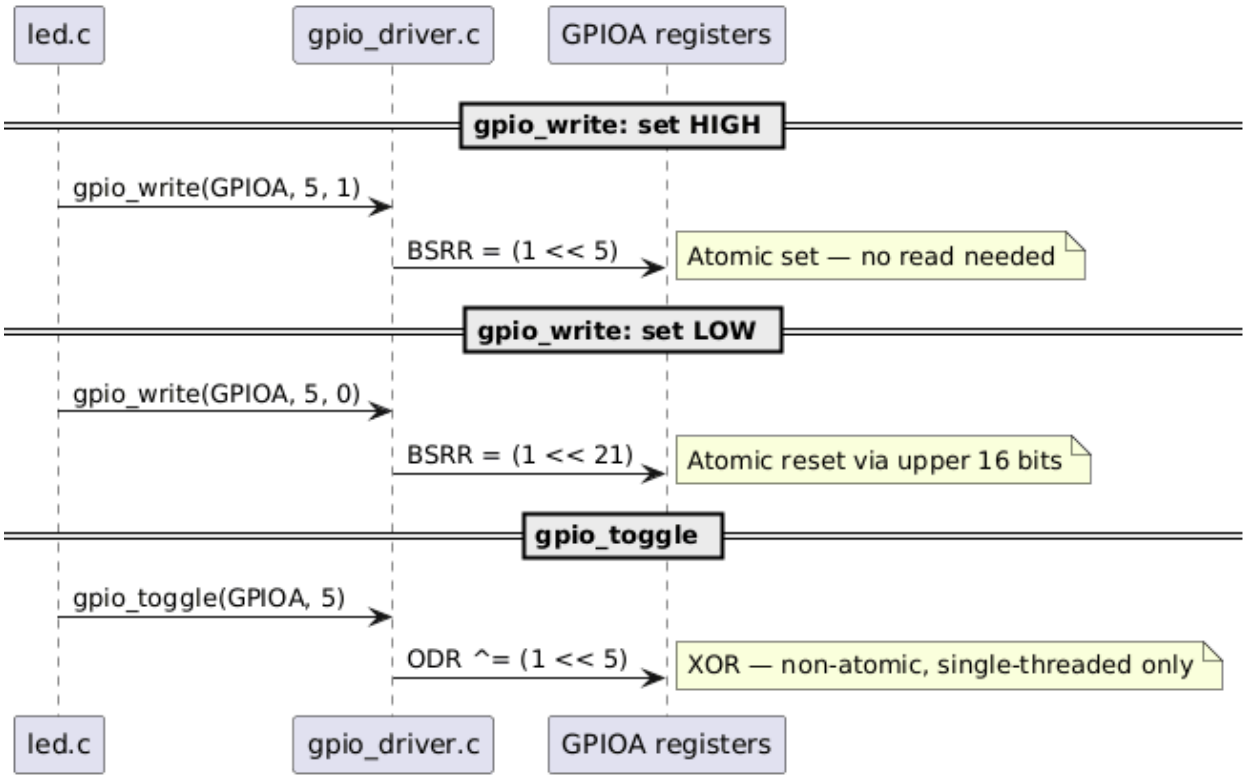

Figure 4. Sequence Diagram — Atomic gpio_write() via BSRR, gpio_toggle() via ODR XOR

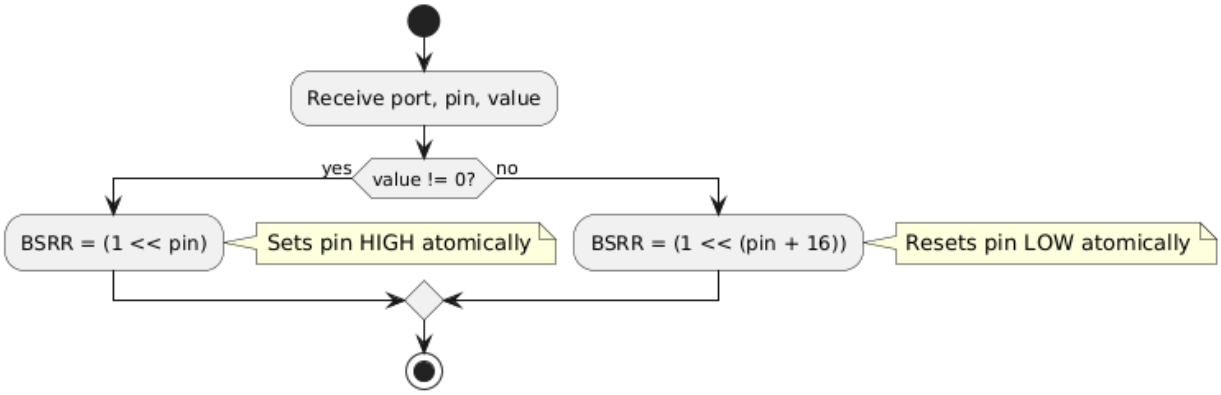

Figure 5. Activity Diagram — gpio_write() BSRR atomic set/reset

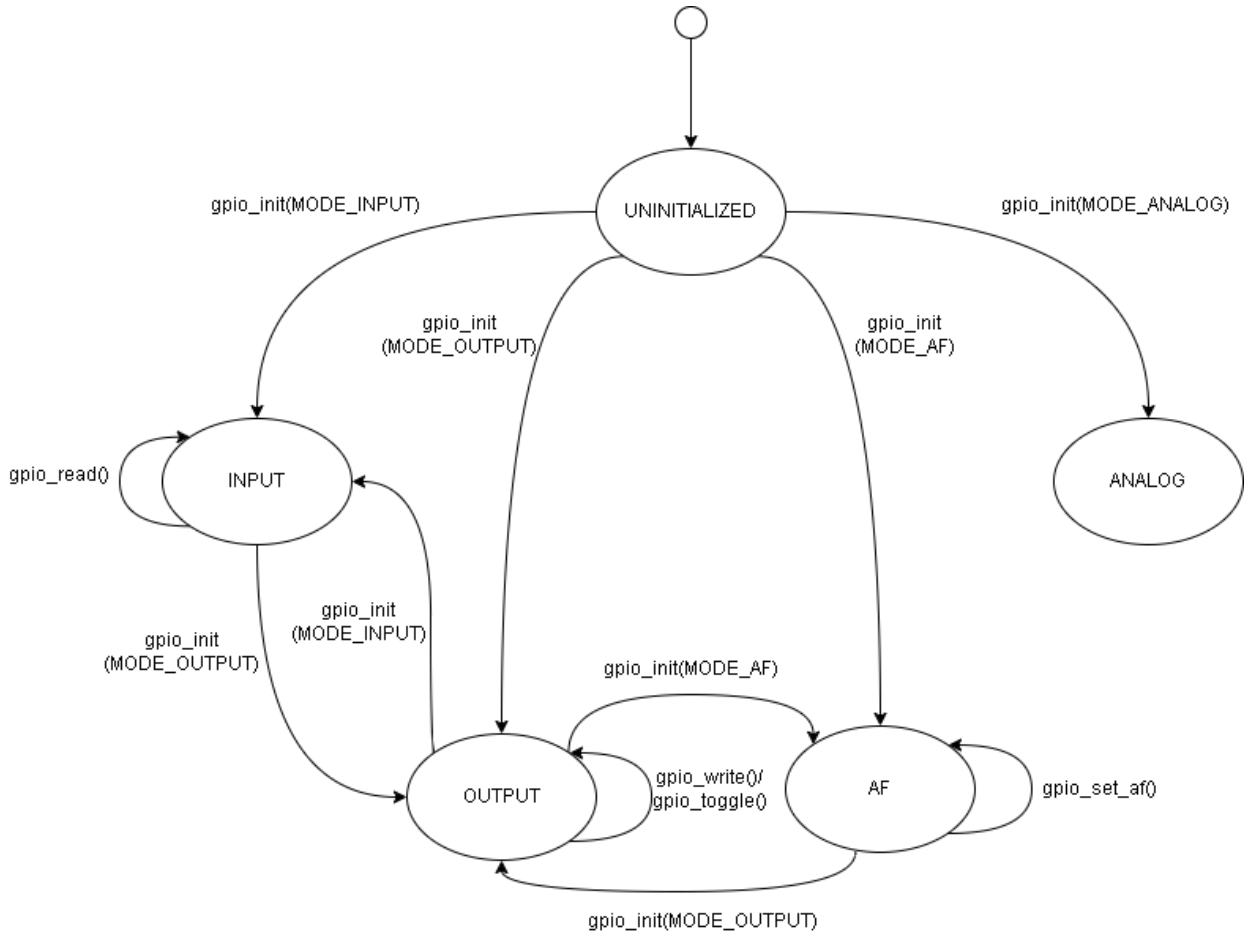

Figure 6. State Diagram — GPIO pin states: UNINITIALIZED → INPUT / OUTPUT / AF / ANALOG

3.3 GPIO Driver Module Description

gpio_init() enables the peripheral clock via the private gpio_enable_clock() helper, validates port and pin, then writes the 2-bit MODER field using a read-modify-write.

gpio_set_output_type(), gpio_set_speed(), and gpio_set_pupd() apply the same read-modify-write pattern to OTYPER (1 bit), OSPEEDR (2 bits), and PUPDR (2 bits) respectively.

gpio_set_af() selects AFRL (AFR[0]) for pins 0–7 or AFRH (AFR[1]) for pins 8–15, writing the 4-bit AF field at the correct bit offset. All functions return GPIO_ERROR if port is NULL or pin > 15.

gpio_write() uses BSRR for atomic operation: writing bit n of BSRR[15:0] sets ODR bit n HIGH; writing bit n of BSRR[31:16] resets it LOW. No read-modify-write is needed.

gpio_toggle() XORs the ODR register directly. This is non-atomic and should only be used in single-threaded contexts.

gpio_read() reads the IDR register and masks the target bit. Returns 1 if HIGH, 0 if LOW.

4. Application Programming Interfaces

4.1 Data Types and Macros

Enum	Values	Notes
GPIO_Mode_t	INPUT=0, OUTPUT=1, AF=2, ANALOG=3	MODER 2-bit field value per pin
GPIO_OType_t	PP=0, OD=1	OTYPER 1-bit field per pin
GPIO_Speed_t	LOW=0, MEDIUM=1, HIGH=2, VHIGH=3	OSPEEDR 2-bit field per pin
GPIO_PUPD_t	NONE=0, PULLUP=1, PULLDOWN=2	PUPDR 2-bit field per pin
GPIO_AF_t	AF0…AF15 (0–15)	AFR 4-bit field per pin
GPIO_Status_t	OK=0, ERROR=1	Return type for all public functions

4.2 Error Codes

Code	Value	Definition
GPIO_OK	0	Operation completed successfully
GPIO_ERROR	1	Invalid argument: port==NULL, pin>15, or AF>15

4.3 Function Definitions

gpio_init

Function Name	gpio_init
Syntax	`GPIO_Status_t gpio_init(GPIO_TypeDef *port, uint8_t pin, GPIO_Mode_t mode)`
Parameters (in)	port — GPIOA…GPIOE pointerpin — 0–15mode — INPUT, OUTPUT, AF, or ANALOG
Parameters (inout)	None
Parameters (out)	None
Return value	GPIO_OK or GPIO_ERROR
Description	Enables GPIO peripheral clock, validates arguments, then writes MODER[2n+1:2n] = mode. Must be called before any other function on the pin.
Constraints	Does not configure OTYPER, OSPEEDR, PUPDR, or AFR — call gpio_set_* functions separately after gpio_init().
Available via	gpio_driver.h

gpio_set_output_type

Function Name	gpio_set_output_type
Syntax	`GPIO_Status_t gpio_set_output_type(GPIO_TypeDef *port, uint8_t pin, GPIO_OType_t otype)`
Parameters (in)	port, pin, otype — PP or OD
Parameters (inout)	None
Parameters (out)	None
Return value	GPIO_OK or GPIO_ERROR
Description	Configures OTYPER[n] = otype. Push-pull can source and sink current; open-drain only sinks.
Constraints	Only meaningful for output-mode pins.
Available via	gpio_driver.h

gpio_set_speed

Function Name	gpio_set_speed
Syntax	`GPIO_Status_t gpio_set_speed(GPIO_TypeDef *port, uint8_t pin, GPIO_Speed_t speed)`
Parameters (in)	port, pin, speed — LOW/MEDIUM/HIGH/VHIGH
Parameters (inout)	None
Parameters (out)	None
Return value	GPIO_OK or GPIO_ERROR
Description	Sets OSPEEDR[2n+1:2n] = speed. Controls output slew rate. Use GPIO_SPEED_HIGH for PWM output pins.
Constraints	Higher speed increases EMI; use the lowest speed sufficient for the application.
Available via	gpio_driver.h

gpio_set_pupd

Function Name	gpio_set_pupd
Syntax	`GPIO_Status_t gpio_set_pupd(GPIO_TypeDef *port, uint8_t pin, GPIO_PUPD_t pupd)`
Parameters (in)	port, pin, pupd — NONE/PULLUP/PULLDOWN
Parameters (inout)	None
Parameters (out)	None
Return value	GPIO_OK or GPIO_ERROR

Description	Configures PUPDR[2n+1:2n]. Enables or disables internal pull resistors (~40 kΩ typical).
Constraints	External resistors take precedence over internal ones.
Available via	gpio_driver.h

gpio_set_af

Function Name	gpio_set_af
Syntax	`GPIO_Status_t gpio_set_af(GPIO_TypeDef *port, uint8_t pin, GPIO_AF_t af)`
Parameters (in)	port, pin (0–15), af (AF0–AF15)
Parameters (inout)	None
Parameters (out)	None
Return value	GPIO_OK or GPIO_ERROR (if af > 15)
Description	Writes the 4-bit AF field in AFR[0] (pins 0–7) or AFR[1] (pins 8–15). For PA8: gpio_set_af(GPIOA, 8, GPIO_AF1) maps to TIM1_CH1.
Constraints	Pin must be in GPIO_MODE_AF via gpio_init() first. Refer to STM32F411 datasheet Table 9 for AF assignments.
Available via	gpio_driver.h

gpio_write

Function Name	gpio_write
Syntax	`void gpio_write(GPIO_TypeDef *port, uint8_t pin, uint8_t value)`
Parameters (in)	port, pin, value — 1=HIGH, 0=LOW
Parameters (inout)	None
Parameters (out)	None
Return value	void
Description	Drives output pin atomically via BSRR. value!=0 → BSRR[pin]=1 (set); value==0 → BSRR[pin+16]=1 (reset). No read-modify-write needed.
Constraints	Pin must be configured as OUTPUT. No NULL/range check (void function).
Available via	gpio_driver.h

gpio_toggle

Function Name	gpio_toggle

Syntax	`void gpio_toggle(GPIO_TypeDef *port, uint8_t pin)`
Parameters (in)	port, pin
Parameters (inout)	None
Parameters (out)	None
Return value	void
Description	Inverts current pin output state via XOR on ODR. Used by led_toggle() in the LED blink loop.
Constraints	Non-atomic (read-modify-write on ODR). Do not use in concurrent contexts.
Available via	gpio_driver.h

gpio_read

Function Name	gpio_read
Syntax	`uint8_t gpio_read(GPIO_TypeDef *port, uint8_t pin)`
Parameters (in)	port, pin
Parameters (inout)	None
Parameters (out)	None
Return value	1 if HIGH, 0 if LOW
Description	Reads current pin state from IDR: (IDR >> pin) & 0x1U.
Constraints	Reads IDR regardless of pin mode.
Available via	gpio_driver.h

4.4 Internal Function Definitions

gpio_enable_clock (static)

Function Name	gpio_enable_clock
Syntax	`static void gpio_enable_clock(GPIO_TypeDef *port)`
Parameters (in)	port — GPIOA…GPIOE pointer
Return value	void
Description	Sets the corresponding GPIOxEN bit in RCC->AHB1ENR. Called automatically at the start of gpio_init(). Not accessible outside gpio_driver.c.
Constraints	Only handles GPIOA–GPIOE. Extend for GPIOH if needed.

5. Future Enhancements

- Add GPIOH support in gpio_enable_clock() for NUCLEO boards that use it.
- Implement gpio_lock() using the LCKR register to protect critical pin configurations.
- Add gpio_set_exti() to configure EXTI interrupt lines for edge-triggered input events.
- Add input validation to gpio_write(), gpio_toggle(), and gpio_read() (currently void functions with no check).